UPES, Dehradun
School of Computer Science · Artificial Intelligence Cluster

Best, Worst and Average Case Analysis

Mid-Semester Assignment Report

Course: Design and Analysis of Algorithms (CSEG3053)
Student: Sparsh Srivastava
SAP ID: 590022251
Batch: B17
Semester: III (Aug–Dec 2026)
Submitted to: Kunwar Siddharth, Assistant Professor
Date: October 12, 2026

Scan to open the animation / code

Abstract

In this assignment, I study how the arrangement of input data directly affects an algorithm's execution time through best, worst, and average case analysis. Focusing on three fundamental algorithms (Linear Search, Insertion Sort, and Quick Sort), I derive their time complexities step by step using basic summations, inversion counting, and the Master Theorem. I also address the common confusion between asymptotic notations (O, Ω, Θ) and input cases. To visualize these concepts, I developed an interactive Python animation using Matplotlib that demonstrates Linear Search and Insertion Sort under all three input scenarios with live operation counters. The report highlights why worst-case analysis remains the primary benchmark for real-time and safety-critical applications.

Contents

1 Introduction **2**

2 Intuition and Core Idea **2**

3 Mathematical Formulation **2**
3.1 Best-Case Complexity . . . 2
3.2 Worst-Case Complexity . . . 3
3.3 Average-Case Complexity . . . 3
3.4 Notation vs. Case: Avoiding Common Confusion . . . 3

4 Algorithm / Procedure **4**
4.1 Linear Search . . . 4
4.2 Insertion Sort . . . 4
4.3 Quick Sort (overview) . . . 5

5 Worked Example (Step by Step) **5**
5.1 Linear Search on $A = [3, 7, 1, 9, 4]$, key $= 9$ . . . 5
5.2 Insertion Sort on $A = [5, 3, 4, 1, 2]$ . . . 5

6 Animation / Software Demonstration **6**

7 Analysis **6**
7.1 Per-Algorithm Case Derivations . . . 6
7.2 Summary Comparison Table . . . 7
7.3 Strengths and Limitations . . . 7

8 Applications **7**

9 Conclusion **8**

Declaration of AI Tool Use **8**

1 Introduction

When studying algorithms in DAA, the first requirement is always correctness, but in practical software development, efficiency is just as important. We usually quantify efficiency by counting primitive operations as a function of the input size n. However, a common mistake is assuming that running time depends strictly on n alone.

In practice, the arrangement and initial order of elements matter just as much. For example, if we search for a book on a shelf containing n unsorted books, we might find it on our very first attempt if we are lucky. But if the book is at the very end or missing altogether, we have to inspect all n books. Even though the problem size is identical, the actual work performed differs by a factor of n.

To evaluate algorithms objectively across different scenarios, we divide our analysis into three cases [1]:

- **Best-case analysis**: Measures the minimum number of steps needed on the most favorable input configuration.
- **Worst-case analysis**: Measures the maximum number of steps taken on the most difficult or adversarial input configuration.
- **Average-case analysis**: Calculates the expected number of steps assuming a realistic probability distribution over all possible inputs.

In this report, I analyze these three cases for Linear Search, Insertion Sort, and Quick Sort, covering linear scan, incremental sorting, and divide-and-conquer strategies.

2 Intuition and Core Idea

Key Idea

For a fixed input size n, the number of operations an algorithm performs can vary dramatically depending on how data elements are arranged. Case analysis captures this variation by calculating the minimum (best), maximum (worst), and expected (average) operation counts.

A simple analogy helps build intuition here. Consider passing through airport baggage screening. If your bag triggers no alarms, you pass through in seconds (the best case). If an item looks suspicious and requires a full manual search of every pocket, you spend several minutes (the worst case). Over a typical day, the security line moves at an average pace determined by normal traveler traffic (the average case).

In computer science, algorithms behave the exact same way. Depending on how the input data is ordered, the program takes different execution paths through its loops and conditional statements. Figure 1 illustrates how a single input size n splits into three separate performance paths.

3 Mathematical Formulation

Let $\mathcal{I}_n$ denote the set of all valid inputs of size n. For any given input $I \in \mathcal{I}_n$, let $T(I)$ be the number of primitive operations (such as comparisons, assignments, or swaps) executed by algorithm $\mathcal{A}$.

3.1 Best-Case Complexity

$$T_{\text{best}}(n) = \min_{I \in \mathcal{I}_n} T(I) \tag{1}$$

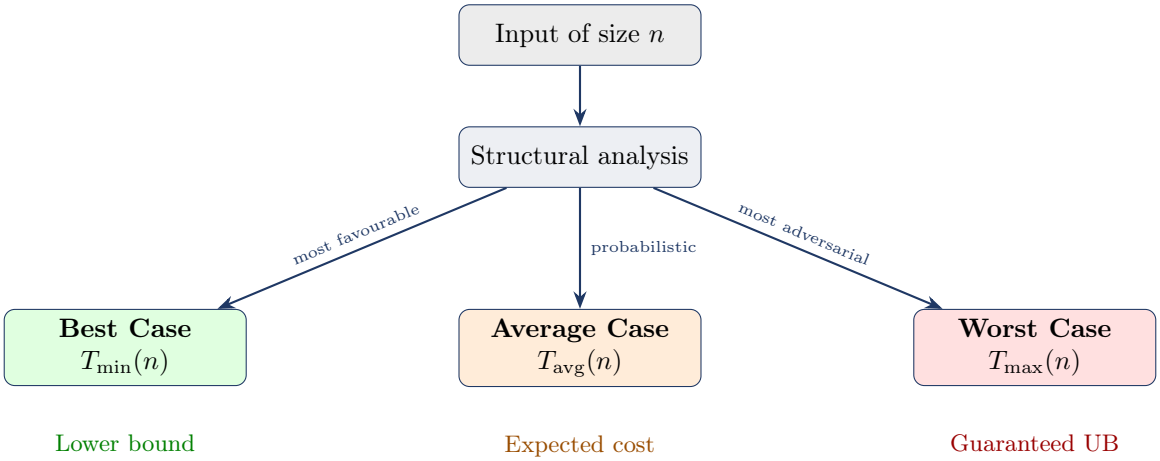


Figure 1: Three cases arising from the same input size n. Each represents a different set of input configurations; together they bound and characterise the full performance envelope of an algorithm.

Equation (1) defines the absolute minimum work an algorithm can do on an input of size n. While it shows whether an early-exit optimization works, it rarely provides a practical guarantee for everyday inputs.

3.2 Worst-Case Complexity

$$T_{\text{worst}}(n) = \max_{I \in \mathcal{I}_n} T(I) \tag{2}$$

Equation (2) gives an upper bound guarantee: no matter what input of size n we feed into the program, it will never execute more than $T_{\text{worst}}(n)$ operations. Because of this certainty, engineers rely on worst-case bounds for safety-critical systems [1].

3.3 Average-Case Complexity

If $P(I)$ is the probability of receiving a specific input I, with $\sum_{I \in \mathcal{I}_n} P(I) = 1$, the expected running time is given by:

$$T_{\text{avg}}(n) = \mathbb{E}[T] = \sum_{I \in \mathcal{I}_n} P(I) \cdot T(I) \tag{3}$$

Assuming every permutation of n elements is equally likely to occur, we get the uniform distribution formula:

$$T_{\text{avg}}(n) = \frac{1}{n!} \sum_{I \in \mathcal{I}_n} T(I) \tag{4}$$

3.4 Notation vs. Case: Avoiding Common Confusion

A frequent conceptual mistake in DAA is treating asymptotic notations (O, Ω, Θ) as direct synonyms for worst, best, and average cases. As shown in Table 1, they answer two completely different questions.

Figure 2 plots the three dominant growth functions that appear in our analysis: n, $n \log n$, and n^2.

Concept	What it answers	Example
Best / Worst / Avg case O, Ω, Θ	*Which inputs are we evaluating?* *What bound applies to $T(n)$?*	Worst case of Quick Sort $\Theta(n^2)$

$\Rightarrow$ Worst-case time of Quick Sort is simultaneously $O(n^2)$, $\Omega(n^2)$, and $\Theta(n^2)$.

$\Rightarrow$ Best-case time of Quick Sort is $\Theta(n \log n)$. Saying "Big-O = worst case" is **incorrect**.

Table 1: Asymptotic notation and input cases are orthogonal dimensions. Any notation can be applied to any case.

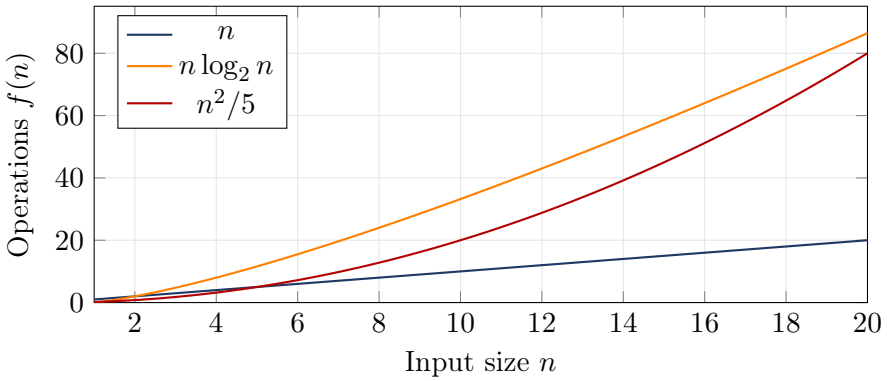


Figure 2: Growth rates of n, $n \log n$, and n^2. Linear Search and Insertion Sort (best case) run in $\Theta(n)$; Quick Sort (average) in $\Theta(n \log n)$; Insertion Sort (worst/average) and Quick Sort (worst) in $\Theta(n^2)$.

4 Algorithm / Procedure

4.1 Linear Search

Algorithm 1: LinearSearch(A, n, k)

Input: Array $A[0 \ldots n-1]$, integer n, target key k
Output: Index i with $A[i] = k$, or -1 if absent

```
for i ← 0 to n − 1 do
    if A[i] = k then
        return i;
return −1;
```

4.2 Insertion Sort

Algorithm 2: InsertionSort(A, n)

Input: Array $A[0 \ldots n-1]$
Output: A sorted in non-decreasing order (in place)

```
for j ← 1 to n − 1 do
    key ← A[j];
    i ← j − 1;
    while i ≥ 0 and A[i] > key do
        A[i + 1] ← A[i];
        i ← i − 1;
    A[i + 1] ← key;
```

4.3 Quick Sort (overview)

Algorithm 3: QuickSort(A, lo, hi)

Input: Array A, indices lo, hi

if $lo < hi$ **then**
  $\pi \leftarrow$ Partition(A, lo, hi) ; // $\Theta(n)$ work at each level
  QuickSort$(A, lo, \pi - 1)$;
  QuickSort$(A, \pi + 1, hi)$;

Core Python excerpt (full source in the repository):

```python
def insertion_sort(A):
    for j in range(1, len(A)):
        key = A[j]
        i = j - 1
        while i >= 0 and A[i] > key:    # inner loop: shift right
            A[i + 1] = A[i]
            i -= 1
        A[i + 1] = key                   # insert key at correct position
    return A
```

5 Worked Example (Step by Step)

5.1 Linear Search on $A = [3, 7, 1, 9, 4]$, key $= 9$

This traces the *average case*: the key is present at index 3.

Step	Index i	$A[i]$	Action
1	0	3	$3 \neq 9$ (no match, increment index)
2	1	7	$7 \neq 9$ (no match, increment index)
3	2	1	$1 \neq 9$ (no match, increment index)
4	3	9	$9 = 9$ (**match found; return index 3**)

Table 2: Linear Search trace: 4 comparisons. Expected value $\frac{n+1}{2} = 3$ for $n = 5$, $p = 1$. This run is slightly above average, consistent with a random position.

5.2 Insertion Sort on $A = [5, 3, 4, 1, 2]$

This traces the *average / general case*.

Pass j	Key	Array before pass	Array after pass	
1	3	[5, 3, 4, 1, 2]	[3, 5, 4, 1, 2]	(1 shift)
2	4	[3, 5, 4, 1, 2]	[3, 4, 5, 1, 2]	(1 shift)
3	1	[3, 4, 5, 1, 2]	[1, 3, 4, 5, 2]	(3 shifts)
4	2	[1, 3, 4, 5, 2]	[1, 2, 3, 4, 5]	(3 shifts)

Table 3: Insertion Sort trace: total 8 shifts. The number of inversions in $[5, 3, 4, 1, 2]$ is also 8, which confirms that each shift eliminates exactly one inversion.

Inversion count verification. The pairs (i, j) with $i < j$ and $A[i] > A[j]$ in the original array are: $(5, 3), (5, 4), (5, 1), (5, 2), (3, 1), (3, 2), (4, 1), (4, 2)$, which gives exactly 8 inversions. □

6 Animation / Software Demonstration

To demonstrate these concepts visually, I built an interactive animation using Python and Matplotlib.

Tool used: Python 3, `matplotlib` (interactive widgets and patches)
Repository link: `https://github.com/SPARSHsrihero1/daa-case-analysis`
How to run:

1. Install dependencies: `pip install -r requirements.txt`
2. Run the program: `python animation.py`

Controls available to the user:

- **Next and Prev buttons**: Step forward or backward through each comparison and shift.
- **Reset button**: Jump back to the initial unsorted state at any time.
- **Best, Avg, and Worst radio buttons**: Switch the input configuration dynamically.
- **Linear Search and Insertion Sort radio buttons**: Switch between the two algorithms.

Instead of performing expensive re-calculations on every button click, the script pre-computes each step into an explicit list of program states. Each state records the array contents, which bars to highlight, the step explanation label, and the updated comparison and shift counters.



(a) Initial array (b) First comparison (c) Shift in progress (d) Sorted complete

Figure 3: Key frames from the animation. Frame (a) shows the unsorted array in blue. Frame (b) highlights the first comparison in orange. Frame (c) shows an element (purple) being shifted during Insertion Sort. Frame (d) shows the fully sorted array in green with the final operation count.

7 Analysis

7.1 Per-Algorithm Case Derivations

Linear Search

Best case. Key at index 0: one comparison. $T_{\text{best}}(n) = 1 = \Theta(1)$.

Worst case. Key absent or at index $n-1$: n comparisons. $T_{\text{worst}}(n) = n = \Theta(n)$.

Average case. Let p be the probability the key is present. If present, it is equally likely at any of the n positions:

$$\mathbb{E}[T(n)] = \frac{p}{n}\sum_{i=1}^{n} i + n(1-p) = \frac{p(n+1)}{2} + n(1-p). \tag{5}$$

For $p = 1$: $\mathbb{E}[T(n)] = \frac{n+1}{2} = \Theta(n)$.

Insertion Sort

Best case. Already sorted; the `while` condition fails immediately each pass: $T_{\text{best}}(n) = \sum_{j=1}^{n-1} 1 = n - 1 = \Theta(n)$.

Worst case. Reverse sorted; pass j executes j shifts:

$$T_{\text{worst}}(n) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} = \Theta(n^2). \tag{6}$$

Average case (inversion analysis [2]). Each shift removes one inversion. A random permutation has expected inversions

$$\mathbb{E}[\text{inv}] = \binom{n}{2} \cdot \frac{1}{2} = \frac{n(n-1)}{4}, \tag{7}$$

giving $T_{\text{avg}}(n) = \Theta(n^2)$.

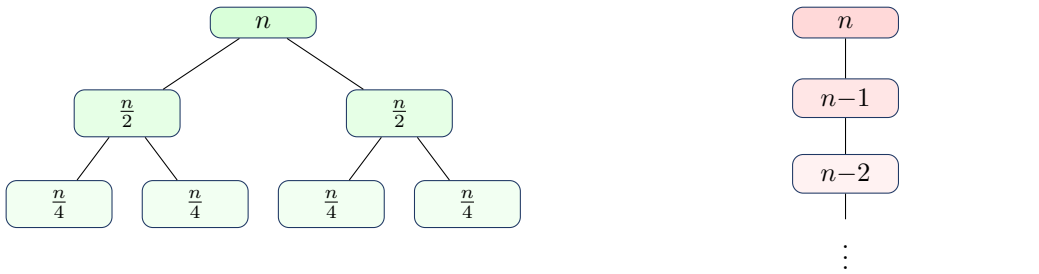
Quick Sort

Best case. Balanced pivot; recurrence $T(n) = 2T(n/2) + \Theta(n)$. By Master Theorem Case 2 ($f(n) = \Theta(n^{\log_2 2})$): $T_{\text{best}}(n) = \Theta(n \log n)$.

Worst case. Degenerate pivot (e.g. sorted array, last-element pivot): $T(n) = T(n-1) + \Theta(n)$. Unrolling: $T(n) = c \sum_{i=1}^{n} i = \Theta(n^2)$.

Average case. Under uniform random permutations [1]: $T_{\text{avg}}(n) = 2n \ln n + O(n) \approx 1.39\, n \log_2 n = \Theta(n \log n)$.

Figure 4 compares the Quick Sort recursion trees for best and worst cases.



Best case: depth $\log_2 n$, $\Theta(n \log n)$ **Worst case: depth n, $\Theta(n^2)$**

Figure 4: Quick Sort recursion trees. The balanced best case (left) has logarithmic depth; the degenerate worst case (right) degrades to a linear chain, giving quadratic total work.

7.2 Summary Comparison Table

7.3 Strengths and Limitations

8 Applications

1. Hard real-time and embedded systems. In safety-critical applications like aircraft flight control, automotive anti-lock brakes (ABS), and biomedical pacemakers, computations must finish before strict deadlines. If an algorithm takes even slightly longer than expected, the delay could cause serious physical failure. In these systems, average-case speed is not enough; engineers must design around proven worst-case execution time (WCET) bounds [3].

2. Guarding against algorithmic complexity attacks (HashDoS). When web applications use hash tables with predictable hash functions, an attacker can intentionally craft requests

Algorithm	Best	Average	Worst	Space
Linear Search	$\Omega(1)$	$\Theta(n)$	$O(n)$	$O(1)$
Binary Search	$\Omega(1)$	$\Theta(\log n)$	$O(\log n)$	$O(1)$
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(1)$

Table 4: Complexity summary. Merge Sort and Heap Sort achieve $\Theta(n \log n)$ in all cases; Quick Sort matches them on average but degrades to $O(n^2)$ in the worst case without randomisation.

Criterion	Worst-case analysis	Average-case analysis
Guarantee	Unconditional upper bound	Requires input distribution model
Practical relevance	May be overly pessimistic	Reflects typical runtime
Analytical difficulty	Often straightforward	Needs combinatorial probability
Use case	Real-time / safety-critical systems	General-purpose applications

Table 5: Comparison of worst-case and average-case analysis paradigms.

containing keys that all collide into the exact same bucket. This forces lookups to degrade from average $O(1)$ time to worst-case $O(n)$ time, exhausting the server's CPU. To prevent this vulnerability, modern software frameworks use randomized universal hashing, which restores expected $O(1)$ performance regardless of what keys an attacker submits [1].

3. Database query optimization. Relational databases like PostgreSQL evaluate whether to perform a full sequential scan ($O(n)$) or use an index scan ($O(\log n)$). The query planner uses average-case cost estimations based on table statistics and filter selectivity to pick the most efficient query plan [4].

9 Conclusion

Studying best, worst, and average case analysis gave me a much clearer perspective on algorithmic trade-offs. The main lesson is that while average-case complexity describes what we encounter most of the time in daily use, worst-case analysis is what guarantees reliability when unusual or adversarial inputs arrive. Furthermore, recognizing that Big-O, Ω, and Θ are mathematical bounding functions rather than synonyms for input cases resolved a common conceptual hurdle.

While creating the step-by-step animation, one observation that stood out was that the number of inner-loop element shifts in Insertion Sort always matches the exact number of inversions in the original input. This clean mathematical connection helped me understand why Insertion Sort runs in $O(n)$ when already sorted, but quickly balloons to $O(n^2)$ when inversions accumulate. A logical next step for exploration would be examining how Shell Sort reduces these inversions across larger gap intervals.

Declaration of AI Tool Use

Tools used: Google Antigravity (AI assistant)
Purpose of use: Assisted with initial LaTeX document setup, table and figure boilerplate, and checking grammar.
Estimated share of AI-generated content: Approximately 10–15% of total assignment effort.

All mathematical derivations, recurrence solving, algorithm traces, Matplotlib animation code (`animation.py`), and personal reflections were independently authored, verified, and debugged

by me.

Signature: _____ Date: _____

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. MIT Press, 4th ed., 2022.

[2] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Addison-Wesley, 2nd ed., 1998.

[3] J. Kleinberg and É. Tardos, *Algorithm Design*. Pearson Addison-Wesley, 2006.

[4] R. Sedgewick and K. Wayne, *Algorithms*. Addison-Wesley, 4th ed., 2011.